

Feature spec — Commitment & procurement optimization

The highest-\$ incremental lever (see `docs/market-analysis-ai-spend-governance.md` §10 and `docs/product-strategy.md` §3B). This specs the **next build after the core**: from Outlay's attributed + forecasted usage, recommend the optimal way to pay for AI compute, pace existing commitments, and arm renewals — **read-only and advisory** (we recommend; the customer executes the commitment with the vendor; we never sit in the request path).

1. Problem & goal

AI vendors price like cloud: **on-demand rack rate**, **committed-spend discounts** (commit \$X → ~10-40% off), and **provisioned throughput / reserved capacity** (Azure OpenAI PTUs, Bedrock provisioned, reserved GPUs — fixed price for a dedicated lane). Committing lowers the **unit** price but the **effective** cost depends on utilization — there's a **break-even**. Companies misjudge it because they can't forecast or attribute usage, so they (a) overpay on-demand, (b) over-commit and forfeit, or (c) under-commit and eat overage.

Goal: turn Outlay's differentiated data (attributed, work-joined, forecasted usage) into three outputs — **(1) a commitment recommendation, (2) commitment pacing, (3) a renewal/negotiation pack** — so a customer lands above break-even without guessing.

Boundaries (preserve the moat/posture): advisory only; metadata-only; no contract documents needed (just the numbers); no routing/enforcement in-path. The cloud analog (ProsperOps/Zesty/nOps) is autonomous execution on cloud infra; we differentiate on the **work-attributed steadiness signal** they don't have, and stay read-only.

2. Inputs (data model)

Entity	Fields	Source
usage_series	account, provider, model, ts, input_tok, output_tok, on_demand_cost, workload_key (team/project/ticket)	Have — extend the existing attribution snapshots to per-provider/model granularity over time
provider_rate	provider, model, on_demand \$/Mtok (in/out), batch_discount, cache_discount	New — maintained rate-card config (versioned)
commitment_tier	provider, threshold \$, discount %	New — rate-card config
provisioned_unit	provider, unit (PTU/GPU), \$/unit/hr, tokens_per_sec_capacity	New — config; PTU pricing is often negotiated → allow customer override
customer_commitment		

Entity	Fields	Source
	account, provider, type (committed_spend provisioned), amount/units, start, end, forfeit_rule, used_to_date	New — customer inputs the numbers (metadata only)
forecast	per-workload / per-provider expected spend over horizon + band	Have — existing forecast engine

Everything customer-specific is **numbers, not documents** — consistent with metadata-only/BYOK.

3. Recommend logic

3a. Baseline-vs-spike decomposition. From `usage_series`, separate the **steady "always-on" floor** (e.g., the p10-p25 of daily/hourly spend over a trailing window) from the **spiky remainder**. The floor is the commit/provision candidate; the spike stays on-demand. Per workload, compute steadiness = $1 - \text{coefficient of variation}$; rank workloads steady→spiky.

3b. Committed-spend discount. Recommend committing the **forecasted floor over the term × (1 – safety buffer)** so utilization stays $\geq \sim 100\%$ of the commit (you must use it to capture the discount). Net benefit = `discount% × (commit you'll actually use) – forfeited unused`. Output the **modeled effective-savings-rate** and **downside (forfeit risk)** at three commit levels (conservative / base / aggressive).

3c. Provisioned throughput break-even. Provisioned beats on-demand only above a utilization threshold **U***:

$$U^* = (\text{provisioned } \$/\text{hr}) / (\text{on_demand } \$/\text{token} \times \text{tokens_per_sec_capacity} \times 3600)$$

Recommend moving the portion of the **steady floor that exceeds U*** to provisioned; keep the rest on-demand. (If PTU price is negotiated/unknown, ask the customer to input their quote.)

3d. Per-workload split (the differentiated output). Because Outlay knows which work is steady (via the attribution join), produce: "Move workloads A, B (steady, $\sim N$ tok/day, CoV < 0.3) to provisioned/committed; leave C, D (spiky) on-demand." This is the recommendation an infra-only optimizer can't make — it's the moat applied to procurement.

3e. Advisory cheaper-equivalents (read-only flags, not in-path). From the data: batch-API candidates (async-tolerant workloads), prompt-caching candidates (repeated-context workloads), cheaper-model candidates (over-provisioned model for the task class). Surface as recommendations with estimated savings — the customer implements them.

4. Pace logic (commitment pacing) — extends shipped program pacing

For each active `customer_commitment` : - Track `used_to_date` vs the expected pace (linear share of term **and** forecast-projected) → project **end-of-term utilization**. - **Forfeit risk** (under-pace): "On pace to use 70% of your \$2M Anthropic commit → ~\$600k forfeited. Shift workload onto it or renegotiate the tier." - **Overage risk** (over-pace): "You'll exhaust the commit in month 8 → on-demand overage on the remainder. Consider a higher tier." - **Status + alerts** reuse the existing budget/pacing rails: on-track (green), watch (amber), off (red), with digest / Slack / SIEM-webhook delivery already built.

5. Surfaces (UI)

- 1. **Commitment recommender** (new page): per provider — recommended posture (commit / provision / on-demand split), modeled savings %, break-even, and downside, with the per-workload split (3d).
- 2. **Commitment pacing** (card on the Governance page): per active commitment — pace gauge + forfeit/ overage projection (reuses the program-pacing component).
- 3. **Renewal / negotiation pack** (export): attributed spend history + forecast + recommended commit for the next term — the artifact the customer takes to the vendor.

6. Reuse of existing primitives (why this is incremental, not a rebuild)

Need	Existing Outlay primitive
Usage time-series by team/ticket	Attribution snapshots (<code>outlay_program_history</code> , per-program spend) — extend to per-provider/model
Floor / term projection	Forecast engine + back-test
Commitment pacing	Program-pacing / earned-value engine (<code>program_pacing</code> , <code>program_earned_value</code>)
Forfeit/overage alerts	Budget-alert + webhook/Slack/digest rails
Read-only posture	Unchanged — recommend only, never route

7. Phasing

- **MVP** — rate-card config + **on-demand vs committed-discount recommender** + **commitment pacing (forfeit/overage)** for **one provider** (Anthropic or Azure OpenAI). Proves the savings story.

- **v2 — provisioned-throughput break-even + per-workload steady/spiky split.**
 - **v3 — multi-provider portfolio optimization** (the "decide at the portfolio level after ≥ 30 days of real traffic" discipline) + **negotiation pack** + advisory cheaper-equivalents.
-

8. Risks / open questions

- **Rate-card maintenance** — provider prices change often; needs a versioned config pipeline (and customer override for negotiated/PTU prices).
- **Forfeit rules vary by contract** — collect as customer-input numbers, not parsed documents.
- **Verification of savings** — like the core coverage number, measure realized savings from pilots rather than claim a benchmark (the market lacks a credible figure — owning a measured one is a moat).
- **Competitive** — Flexera (ProsperOps + FinOps-for-AI) is the most likely to extend into this; the defense is the **work-attributed steadiness signal** + read-only posture they don't lead with. Ship while it's a gap.